

Data Preparation for Business Analytics

Mock Exam — Questions & Detailed Solutions

April 2026

This document contains all 18 mock exam questions with full worked solutions, explanations of common mistakes, and key concepts to remember. Questions cover Sessions 1–9: data import, reshaping, missing values, duplicates, outliers, dates, string manipulation, and regular expressions.

Question 1: What is wrong with the following code? How would you fix it?

```
df <- data.frame(A = c(1, 2, NA, 4), B = c(5, 6, 7, 8)) df <- filter(df, !is.na(A | B))
```

✓ ANSWER — Logical operator misplaced inside is.na()

The bug is that `A | B` is evaluated **before** `is.na()`. The expression `is.na(A | B)` first computes the logical OR of A and B (which itself becomes NA when A is NA), and only then checks for NA — so it does not correctly test whether *either column* is missing.

There are two meaningful corrections depending on intent:

- **Keep rows where A is not NA** (the most likely intent — filter out rows with missing A):

```
df <- filter(df, !is.na(A))
```

- **Keep rows where neither A nor B is NA** (remove any row with any missing value):

```
df <- filter(df, !is.na(A) & !is.na(B))
```

- **Keep rows where at least one of A or B is not NA:**

```
df <- filter(df, !is.na(A) | !is.na(B))
```

■ `is.na()` must be applied to each column individually. `is.na(A | B)` is syntactically valid but logically wrong — it evaluates the OR first, producing a single logical value or NA, then checks that for NA-ness.

Question 2: What is wrong with this regular expression for extracting email addresses? Fix the mistake.

```
emails <- str_extract_all(text, "[a-zA-Z0-9_.]+"@"[a-zA-Z_]+\\\\". [a-zA-Z]+")
```

✓ ANSWER — Domain part does not allow digits or hyphens

The pattern `[a-zA-Z_]+` for the domain name only permits letters and underscores. Real domain names also contain **digits** and **hyphens** (e.g. `my-company2.com`, `gmail.com`). Additionally, the TLD (top-level domain) part after the dot should not allow underscores and must be at least 2 characters.

Fixed pattern:

```
# Correct version – allows digits and hyphens in domain, proper TLD: emails <- str_extract_all(text, "[a-zA-Z0-9_+.-]+"@"[a-zA-Z0-9-]+"\\\\". [a-zA-Z]{2,}+")
```

Changes made:

- Local part: added `+-` (common in email local parts)
- Domain: changed `[a-zA-Z_]+` to `[a-zA-Z0-9-]+` (digits + hyphens, no underscore)
- TLD: changed `[a-zA-Z]+` to `[a-zA-Z]{2,}` (minimum 2 chars, no underscores)

■ In R strings, each regex backslash requires double-escaping: `\\` in the string becomes `\` in the regex, which matches a literal dot.

Question 3: Fill in the missing function to replace all missing values in column Price with the mean of that column:

```
df$Price[is.na(df$Price)] <- _____(df$Price, na.rm = TRUE)
```

✓ ANSWER — mean()

```
df$Price[is.na(df$Price)] <- mean(df$Price, na.rm = TRUE)
```

The function is `mean()`. The argument `na.rm = TRUE` tells R to ignore existing NAs when computing the mean — without it, any NA in the column would make the result NA, and the imputation would replace NAs with NA (no effect).

The statement works in two steps:

- `is.na(df$Price)` creates a logical index of which positions are NA
- The assignment `<-` replaces only those positions with the computed mean

■ Mean imputation is simple but has drawbacks: it reduces variance and can distort relationships between variables. It is appropriate when data is missing at random and the proportion of missing values is small (typically < 5–10%).

Question 4: Match the regular expression pattern to its purpose.

Regex Pattern	Purpose	Answer
<code>\\d+</code>	(a) Extract email addresses	→ (b)
<code>\\w+@\\w+\\.\\w+</code>	(b) Extract numbers	→ (a)
<code>[A-Z]{3}</code>	(c) Find three-letter uppercase codes	→ (c)

✓ ANSWER — `\\d+` → (b) | `\\w+@\\w+\\.\\w+` → (a) | `[A-Z]{3}` → (c)

Explanation of each pattern:

- `\\d+` — matches one or more digits (0–9). This is the standard way to **extract numbers** from text → (b)
- `\\w+@\\w+\\.\\w+` — matches word characters, then @, then word characters, then a literal dot, then word characters. This is a simplified **email address** pattern → (a)
- `[A-Z]{3}` — matches exactly 3 consecutive uppercase letters (A–Z). Used to find **three-letter uppercase codes** like country codes (FRA, USA) or airport codes → (c)

Question 5: Match each R function to the type of data it is used to load.

Function	Data Type	Answer
A. <code>read.csv()</code>	1. JSON File	→ 2
B. <code>fromJSON()</code>	2. CSV File	→ 1
C. <code>readRDS()</code>	3. Excel File	→ 4
D. <code>read_excel()</code>	4. R Serialized Object	→ 3

✓ ANSWER — A→2, B→1, C→4, D→3

Explanation:

- `read.csv()` (base R) / `read_csv()` (tidyverse readr) → reads **CSV files** → (2)
- `fromJSON()` from the `jsonlite` package → reads **JSON files** → (1)
- `readRDS()` → reads **R serialized objects** (.rds files) — R's native binary format that preserves all object attributes → (4)
- `read_excel()` from the `readxl` package → reads **Excel files** (.xlsx, .xls) → (3)

■ *readxl is installed with tidyverse but must be loaded separately: `library(readxl)`*

Question 6: Match each function to its correct purpose.

Function	Purpose	Answer
A. pivot_longer()	1. Convert wide format to long format	→ 1
B. pivot_wider()	2. Convert long format to wide format	→ 2
C. separate()	3. Split a column into multiple columns	→ 3
D. unite()	4. Merge multiple columns into one	→ 4

✓ ANSWER — A→1, B→2, C→3, D→4

Explanation:

- `pivot_longer()` → takes multiple columns and collapses them into two columns (name + value). Makes data **longer** (more rows, fewer columns). Use when column headers are actually values (e.g. income brackets as column names).
- `pivot_wider()` → spreads a key-value pair across multiple columns. Makes data **wider** (fewer rows, more columns). Use when one column stores multiple variable names.
- `separate()` → splits one column into multiple columns using a delimiter or fixed position. E.g. splitting "287 - CELLULITIS" into DRGcode and DRGdescription.
- `unite()` → combines multiple columns into one, with an optional separator. E.g. merging City and State into "Chicago, IL".

Question 7: Match the method with the correct way it identifies outliers.

Method	Description	Answer
A. <code>boxplot.stats(df\$column)\$out</code>	1. Identifies outliers using the IQR method	→ 1
B. <code>abs(scale(df\$column)) > 3</code>	2. Identifies outliers based on standard deviation (z-score)	→ 2
C. <code>df %>% filter(col > mean + 3*sd)</code>	3. Filters extreme values beyond 3 SDs from the mean	→ 3
D. <code>quantile(df\$col, c(0.05, 0.95))</code>	4. Finds 5th and 95th percentile to detect outliers	→ 4

✓ ANSWER — A→1, B→2, C→3, D→4

Explanation:

- **A — `boxplot.stats()`\$out → IQR method (1):** Boxplot uses the IQR rule: outliers are points beyond $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ (the whisker boundaries). This is a non-parametric, distribution-free method.
- **B — `abs(scale())` > 3 → z-score method (2):** `scale()` standardises the column to mean=0, SD=1. Taking the absolute value and comparing to 3 identifies points more than 3 standard deviations from the mean. This assumes an approximately normal distribution.
- **C — filter with mean + 3*sd → 3-SD filter (3):** Functionally equivalent to B but written as an explicit filter, keeping only the extreme upper tail. Note: B returns a logical vector for all rows (both tails); C as written only captures the upper tail.
- **D — `quantile(0.05, 0.95)` → percentile method (4):** Returns the actual data values at the 5th and 95th percentiles. Values outside these bounds are flagged as potential outliers. More robust than SD-based methods for skewed distributions.

■ The IQR (boxplot) method is the most commonly used and most robust to skewness. SD-based methods assume normality. The percentile method is flexible and intuitive.

Question 8: You receive a dataset df with duplicate rows. How would you remove duplicates while keeping only the first occurrence?

✓ ANSWER — Use `duplicated()` or `dplyr::distinct()`

Method 1 — Base R with `duplicated()`:

```
# duplicated() returns TRUE for each row that is a duplicate of an earlier row #  
Negating with ! keeps only the first occurrence of each row df <-
```

```
df[!duplicated(df), ]
```

Method 2 — tidyverse distinct():

```
# distinct() keeps unique rows, by default the first occurrence df <- df %>%  
distinct() # To deduplicate based on specific columns only: df <- df %>%  
distinct(ID, Name, .keep_all = TRUE)
```

Method 3 — unique() (for exact row duplicates):

```
df <- unique(df) # removes exact duplicate rows
```

Key difference between methods:

- duplicated() and unique() work on exact row matches across all columns
- distinct(.keep_all=TRUE) allows deduplication on a subset of columns while retaining all columns

■ Always inspect what you consider a "duplicate" before removing. Two rows with the same name but different IDs may be different people, not duplicates!

Question 9: A dataset contains timestamp data in different time zones. Explain how you would standardize all timestamps to UTC in R.

✓ **ANSWER — Use `lubridate::with_tz()` to convert to UTC**

The key distinction is between **converting** the timezone (the moment in time stays the same, the display changes) and **forcing** a timezone (you assert that the stored value is in a specific zone, without shifting the clock).

Step 1 — Ensure timestamps are parsed with their correct timezone:

```
library(lubridate) # Parse timestamps that you know are in specific timezones:
ts_paris <- ymd_hms("2026-04-13 09:00:00", tz = "Europe/Paris") ts_ny <-
ymd_hms("2026-04-13 03:00:00", tz = "America/New_York") ts_tokyo <-
ymd_hms("2026-04-13 16:00:00", tz = "Asia/Tokyo")
```

Step 2 — Convert all to UTC using `with_tz()`:

```
# with_tz() converts: the underlying moment stays the same, the display changes
ts_paris_utc <- with_tz(ts_paris, "UTC") ts_ny_utc <- with_tz(ts_ny, "UTC")
ts_tokyo_utc <- with_tz(ts_tokyo, "UTC") # All three will show the same UTC time:
2026-04-13 07:00:00 UTC
```

Step 3 — Apply to an entire column in a data frame:

```
# If all timestamps are stored without timezone info but you know they are in Paris:
df <- df %>% mutate(timestamp = force_tz(timestamp, "Europe/Paris")) %>% # assign
tz mutate(timestamp = with_tz(timestamp, "UTC")) # convert to UTC
```

`force_tz()` vs `with_tz()`:

- `with_tz(x, tz)` — converts the time: "what time is it in UTC when it is 9am in Paris?"
- `force_tz(x, tz)` — relabels the timezone without shifting: "this stored value IS in Paris time"

■ Use `OlsonNames()` to find valid timezone strings. E.g. `grep("Paris", OlsonNames(), value=TRUE)` returns "Europe/Paris".

Question 10: What will be the result of the following filtering operation?

```
df <- data.frame(A = c(10, 15, 20, 25), B = c(5, 7, 10, 12)) df_filtered <-
filter(df, A > 15 & B < 10) print(df_filtered)
```

✓ **ANSWER — An empty data frame (0 rows)**

Let's evaluate the condition `A > 15 & B < 10` for each row:

Row	A	B	A > 15	B < 10	A > 15 & B < 10
1	10	5	FALSE	TRUE	FALSE
2	15	7	FALSE	TRUE	FALSE
3	20	10	TRUE	FALSE	FALSE
4	25	12	TRUE	FALSE	FALSE

No row satisfies *both* conditions simultaneously. The rows where `A > 15` (rows 3 and 4) have `B ≥ 10`, so `B < 10` is FALSE. The rows where `B < 10` (rows 1 and 2) have `A ≤ 15`, so `A > 15` is FALSE. The result is an **empty data frame with 0 rows and 2 columns**.

```
# Output: [1] A B <0 rows> (or 0-length row.names)
```

■ **A common trap: using `|` (OR) instead of `&` (AND).** With OR, rows 1-4 would all be included.

Question 11: Predict the output of this R command:

```
str_extract_all("My number is 123-456-7890", "\\d{3,}")
```

✓ **ANSWER — A list containing: c("123", "456", "7890")**

The pattern `\\d{3,}` (in R string notation) becomes `\d{3,}` in the actual regex, meaning: **"one or more digits, with at least 3"**.

The input string "My number is 123-456-7890" contains three groups of digits separated by hyphens. Each hyphen breaks the match, so the regex finds them as separate tokens:

Match	Value	Length	Passes <code>\d{3,}</code> ?
"123"	123	3 digits	YES — exactly 3
"456"	456	3 digits	YES — exactly 3
"7890"	7890	4 digits	YES — $4 \geq 3$

```
# Output (str_extract_all returns a list): [[1]] [1] "123" "456" "7890"
```

■ `str_extract_all()` always returns a list (one element per input string). `str_extract()` returns only the first match as a character vector.

Question 12: What does this command return?

```
mean(c(10, 20, 30, NA), trim = 0.2, na.rm = TRUE)
```

✓ **ANSWER — 20**

Step-by-step evaluation:

- **na.rm = TRUE:** removes NA first. Remaining values: {10, 20, 30}
- **trim = 0.2:** trims 20% from each end of the sorted vector. With 3 values, $20\% \times 3 = 0.6 \rightarrow$ rounds down to 0 values trimmed from each end. All three values remain: {10, 20, 30}
- **mean({10, 20, 30})** = $(10 + 20 + 30) / 3 = 60 / 3 = 20$

```
# Result: [1] 20
```

■ The `trim` argument requires trimming at least 1 full observation per side. With only 3 values, `trim=0.2` trims `floor(0.2×3)=0` values. With 5 values (after NA removal), `trim=0.2` would trim 1 value from each end.

Question 13: Write a regex pattern to find dates in either YYYY-MM-DD or DD/MM/YYYY format.

✓ **ANSWER — Alternation pattern using |**

```
# In R string notation (double backslashes): pattern <-  
"\\d{4}-\\d{2}-\\d{2}|\\d{2}/\\d{2}/\\d{4}" # Usage: str_extract_all(text,  
"\\d{4}-\\d{2}-\\d{2}|\\d{2}/\\d{2}/\\d{4}")
```

Breaking down the pattern:

Part	Regex	Matches
YYYY-MM-DD	<code>\d{4}-\d{2}-\d{2}</code>	4 digits, dash, 2 digits, dash, 2 digits
Alternation	<code> </code>	Match either pattern on the left OR the right
DD/MM/YYYY	<code>\d{2}/\d{2}/\d{4}</code>	2 digits, slash, 2 digits, slash, 4 digits

■ This pattern matches the format syntactically but does not validate logical date ranges (e.g. month 13 or day 45 would pass). For strict validation, additional logic is needed.

Question 14: Write a regex pattern to extract website URLs that start with http:// or https://.

✓ **ANSWER — Use https? to optionally match the "s"**

```
# Pattern: pattern <- "https?://\\S+" # Usage: str_extract_all(text, "https?://\\S+")
```

Breaking down the pattern:

- http — literal characters "http"
- s? — the letter "s" is optional (0 or 1 times), matching both http:// and https://
- :// — literal "://"
- \\S+ — one or more non-whitespace characters (the URL path, query, etc.)

```
# Examples matched: # "http://example.com" #  
"https://www.google.com/search?q=r+regex" # "https://claude.ai" # More precise  
version (stops at punctuation that ends a sentence): pattern <-  
"https?://[^\s,;>\\"]+"
```

■ \\S+ is greedy and may capture trailing punctuation. For cleaner extraction, use a negated character class like [^\s<>"]+ to stop at whitespace or HTML delimiters.

Question 15: Write a regex pattern to extract all words starting with a capital letter from a text string.

✓ **ANSWER — [A-Z][a-z]+ or [A-Z]\w***

```
# Option 1: Capitalised word (uppercase + one or more lowercase letters) pattern <-  
"[A-Z][a-z]+" # Option 2: Word starting with uppercase (any subsequent characters)  
pattern <- "[A-Z]\w*" # Usage: str_extract_all("The Quick Brown Fox jumps",  
"[A-Z][a-z]+" ) # [[1]] [1] "The" "Quick" "Brown" "Fox"
```

Comparing the two options:

Pattern	Matches	Misses
[A-Z][a-z]+	"Paris", "France", "Claude"	ALL-CAPS words like "USA"
[A-Z]\w*	"Paris", "USA", "R2D2"	Nothing — very broad

■ Choose [A-Z][a-z]+ for typical proper nouns/names. Use [A-Z]\w* if you also need to capture abbreviations or codes like "USA" or "GDP".

Question 16: A dataset contains missing values in multiple columns. Describe two approaches to handle them and explain when each approach is appropriate.

✓ **ANSWER — Deletion vs. Imputation — choose based on pattern and importance**

Approach 1: Listwise Deletion (remove rows with missing values)

```
# Remove all rows that have ANY NA: df_clean <- df %>% filter(complete.cases(df)) #  
Or in base R: df_clean <- na.omit(df)
```

- Remove all rows containing at least one NA across any column
- **When appropriate:** missing values are few (< 5%), missing completely at random (MCAR), and the rows with missing data are not systematically different from the rest
- **Risk:** Loses information; can bias results if data is not missing at random

Approach 2: Imputation (replace with estimated values)

```
# Mean imputation (for numerical columns): df$Price[is.na(df$Price)] <-  
mean(df$Price, na.rm = TRUE) # Median imputation (more robust to outliers):  
df$Age[is.na(df$Age)] <- median(df$Age, na.rm = TRUE) # Mode imputation (for  
categorical variables): mode_val <- names(sort(table(df$Category),  
decreasing=TRUE))[1] df$Category[is.na(df$Category)] <- mode_val #  
Conditional/group imputation: df <- df %>% group_by(Grade) %>% mutate(Age =  
ifelse(is.na(Age), mean(Age, na.rm=TRUE), Age))
```

- **When appropriate:** larger proportion of missing data, or when missingness is informative (e.g. absence of a value follows a pattern), or when losing rows would severely reduce sample size

- **Risk:** Mean/median imputation reduces variance and can distort relationships; more sophisticated methods (regression, multiple imputation) exist but are outside this course

■ *Always first investigate WHY values are missing (MCAR, MAR, or MNAR). The strategy depends heavily on the mechanism of missingness.*

Question 17: You have a dataset with Sales values that contain extreme outliers. Describe a robust way to handle them without removing data.

✓ **ANSWER — Winsorization (capping) — replace extremes with boundary values**

The most robust approach that preserves all observations is **Winsorization** (also called capping or clipping): replace outlier values with the nearest acceptable boundary value, rather than removing the row or setting to NA.

```
# Step 1: Define the acceptable range using percentiles lower <- quantile(df$Sales, 0.05, na.rm = TRUE) # 5th percentile upper <- quantile(df$Sales, 0.95, na.rm = TRUE) # 95th percentile # Step 2: Cap values outside the range df <- df %>% mutate(Sales = ifelse(Sales < lower, lower, Sales)) %>% mutate(Sales = ifelse(Sales > upper, upper, Sales)) # Alternative: use IQR-based boundaries (like boxplot whiskers) Q1 <- quantile(df$Sales, 0.25, na.rm = TRUE) Q3 <- quantile(df$Sales, 0.75, na.rm = TRUE) IQR <- Q3 - Q1 lower_iqr <- Q1 - 1.5 * IQR upper_iqr <- Q3 + 1.5 * IQR df <- df %>% mutate(Sales = pmax(Sales, lower_iqr)) %>% mutate(Sales = pmin(Sales, upper_iqr))
```

Why this is robust:

- All rows are retained — no data loss, preserves sample size
- Extreme values no longer dominate means, regressions, or other analyses
- The *rank* of observations is preserved (order unchanged)
- The analyst can still see that a capped value existed (it is at the boundary, not missing)

Alternative robust approaches (without removal):

- **Log transformation:** `df$Sales <- log(df$Sales)` — compresses the scale, reduces the influence of extreme values
- **Use robust statistics:** Replace mean with median, use median absolute deviation (MAD) instead of SD in analyses

■ *Whether to cap, transform, or keep outliers depends on domain knowledge. A "\$9 million White House salary" is clearly erroneous and should be set to NA. A genuinely extreme but correct value (Paris population vs other French cities) should be kept.*

Question 18: Our dataset is about house prices in a small city near the sea. How many outliers are among the houses with sea view? [See boxplot in exam]

✓ **ANSWER — 4 outliers among houses with sea view**

Reading the boxplot labelled "House Prices without and with Sea View":

- **No Sea View (left box, "No"):** Two outlier points are visible above the upper whisker, both near the 750–800 range. These are the two dots above the "No" boxplot.
- **Sea View (right box, "Yes"):** Four outlier points are visible above the upper whisker — one near 750 and three clustered around 600–650 (visible as dots above the whisker of the "Yes" boxplot).

The boxplot uses the standard IQR rule: outliers are any points beyond $Q3 + 1.5 \times IQR$ (upper whisker) or below $Q1 - 1.5 \times IQR$ (lower whisker). Counting the dots above the whisker on the **Yes (sea view) side: 4 outliers.**

```
# How to produce this analysis in R: boxplot(Price ~ Sea, data = df, main = "House Prices without and with Sea view", xlab = "Sea", ylab = "Price") # Count outliers for sea view houses: sea_view <- df %>% filter(Sea == "Yes") outliers <- boxplot.stats(sea_view$Price)$out length(outliers) # → 4
```

■ *When reading a boxplot: the box spans $Q1$ to $Q3$ (IQR), the middle line is the median, whiskers extend to the most extreme value within $1.5 \times IQR$, and individual dots beyond the whiskers are outliers.*

Summary Answer Key

Q	Answer / Key Point
1	Bug: <code>is.na(A B)</code> . Fix: <code>filter(!is.na(A) & !is.na(B))</code> or <code>filter(!is.na(A))</code>
2	Domain <code>[a-zA-Z_]+</code> misses digits/hyphens. Fix: <code>[a-zA-Z0-9_+]+@[a-zA-Z0-9_+]\.[a-zA-Z]{2,}</code>
3	<code>mean()</code> — fill blanks with: <code>mean(df\$Price, na.rm = TRUE)</code>
4	<code>\d+</code> → (b) numbers <code>\w+@\w+\.\w+</code> → (a) email <code>[A-Z]{3}</code> → (c) 3-letter codes
5	A→2 CSV B→1 JSON C→4 R Serialized D→3 Excel
6	A→1 wide-to-long B→2 long-to-wide C→3 split col D→4 merge cols
7	A→1 IQR/boxplot B→2 z-score C→3 3-SD filter D→4 percentile
8	<code>df[!duplicated(df),]</code> OR <code>df %>% distinct()</code> — keep first occurrence
9	<code>with_tz(timestamp, "UTC")</code> after ensuring correct source TZ with <code>force_tz()</code>
10	Empty data frame — no row satisfies <code>A>15 AND B<10</code> simultaneously
11	<code>[[1]] [1] "123" "456" "7890"</code> — <code>\d{3,}</code> matches each digit group
12	20 — NA removed, then <code>trim=0.2</code> trims 0 values (too few), <code>mean(10,20,30)=20</code>
13	<code>"\\d{4}-\\d{2}-\\d{2}\\d{2}\\d{2}\\d{4}"</code>
14	<code>"https?://\\S+"</code> — <code>s?</code> makes the "s" optional
15	<code>"[A-Z][a-z]+"</code> for proper nouns; <code>"[A-Z]\\w*"</code> to include abbreviations
16	Deletion (few random NAs) vs Imputation (many/patterned NAs); choose by MCAR/MAR/MNAR
17	Winsorization: cap at percentile boundaries (e.g. 5th/95th); preserves all rows
18	4 outliers among sea-view houses (dots above upper whisker of "Yes" boxplot)